



Stock Data Management System

Python CLI CRUD Application

Individual Portfolio Project — Data Science 2025

Ahmad Faik Setiawan · JCDS 2904, Purwadhika Digital Technology School





TABLE OF CONTENTS

What's Inside This Report

01	About the Author	03
02	Background	04
03	Problem Statement & Goals	05
04	Scope, Data & Architecture	06
05	CRUD Features (Read · Create · Update · Delete)	09
06	Conclusion & Recommendations	16



ABOUT THE AUTHOR

Ahmad Faik Setiawan

JCDS 2904 — Purwadhika Digital Technology School · Individual Project

Built this project to practice end-to-end CRUD logic in plain Python: modeling stock data as structured records, enforcing input validation at every entry point, separating public and admin access, and making deletion a reversible action rather than a destructive one.

Python

Programming Logic

Data Structures

CLI / UX Design



Ahmad Faik Setiawan

Data Science Student

ahmadfaiksetiawan@gmail.com



BACKGROUND

Why a CRUD System for Stock Data?

Investors and market operators need a fast, tidy way to record and review stock price movements — opening and closing prices, percentage change, and transaction value — without wrestling with spreadsheets.

- Manual record-keeping is error-prone: no input validation, no access control, and a mistaken deletion is gone for good.
- A command-line CRUD system is a focused way to practice the core logic — data modeling, validation, and state management — before scaling up to a database or GUI.
- Every screen in the program is built around one shared dataset (`dataSaham`), so Create, Read, Update and Delete all operate consistently on the same records.

```
$ python "Manajemen Data Saham.py"
=====
      SISTEM MANAJEMEN DATA SAHAM KONOHA
=====

[1] Menampilkan Data Saham
[2] Keluar
[3] Login admin

⇒ Pilih menu yang ingin Anda akses : _
```



PROBLEM STATEMENT

“How can stock market data be organized into a structured CRUD system — with role-based access, layered input validation, and recoverable deletion — implemented entirely as a Python command-line application?”

GOALS

What This Project Set Out to Do

- Build a complete **CRUD flow** — Create, Read, Update, Delete — around one shared stock dataset.
- Implement **role-based access**, separating a public Investor view from authenticated Admin actions.
- Enforce **layered input validation** so invalid or malformed data is never accepted.
- Support **soft delete with a recycle bin** plus full restore, so no data is lost by accident.
- Present every result through a **clean, consistent terminal interface** (PrettyTable + formatted menus).



SCOPE

Two Access Levels, One Dataset

Role	Menu Options
Investor no login required	View Data (all / by ticker / by opening or closing price) · Exit · Login as Admin
Admin password-gated	View Data · Add Data · Update Data · Delete Data · Exit

Login Flow

Investor menu → option [3] Login admin



Enter password → `admin()`



Match → Admin menu | No match → back to Investor menu

Demo credential only (a single shared password). A production version would replace this with per-user, hashed credentials — see Recommendations.



DATA STRUCTURE & TECH STACK

A List of Dictionaries as the Data Store

Key	Type	Description
<code>kode</code>	str	Ticker code, 4 letters (e.g. <code>BBCA</code>)
<code>perusahaan</code>	str	Company name
<code>hargaPembukaan</code>	int	Opening price
<code>hargaPenutupan</code>	int	Closing price
<code>perubahan</code>	float	Percentage price change
<code>nilai</code>	int	Transaction value (IDR)
<code>hapus</code>	bool	Soft-delete flag (True = hidden from view)

Python 3

PrettyTable

List of Dictionary

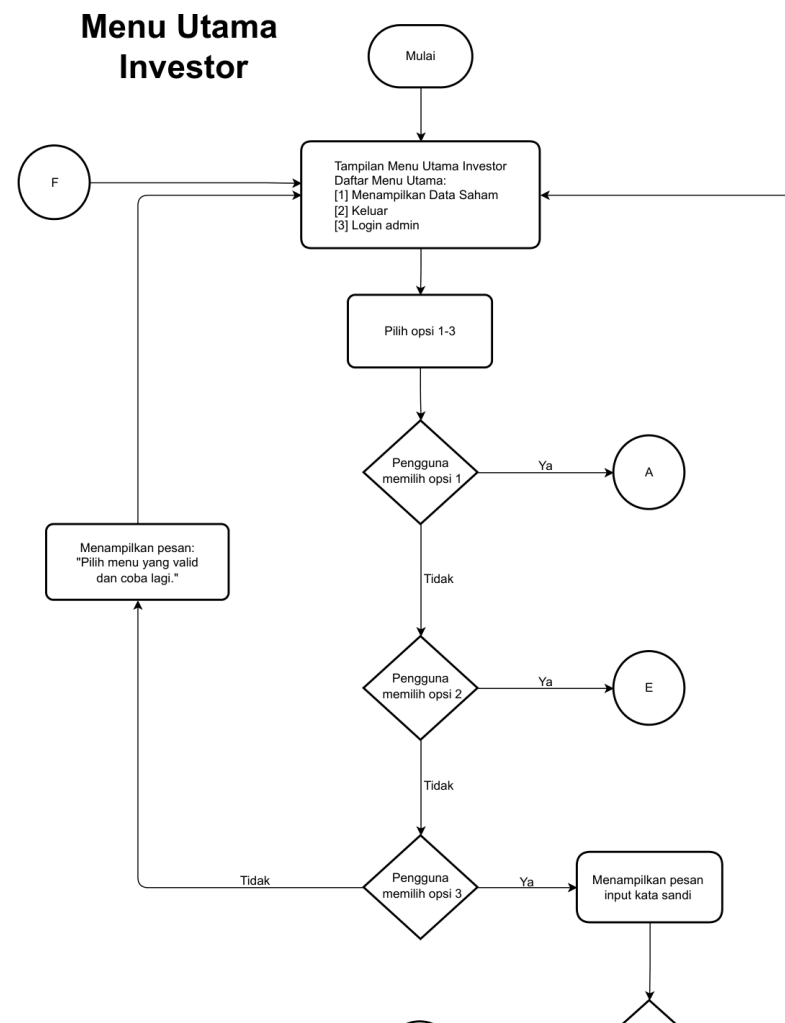
Function-based Programming

Seed dataset: 10 tickers — 7 IDX-listed (AADI, AALI, BBCA, BBRI, BRIS, BMRI, TLKM) + 3 global (AAPL, AMZN, TSLA). Support structure `recycle_bin = []` holds soft-deleted records.



ARCHITECTURE

Program Flow — Investor Main Menu



Reading the Flow

- 1 **Main Menu** renders, the role (Investor / Admin) decides which options appear.
- 2 User **selects a sub-menu** (View, Add, Update, Delete or Login).
- 3 The sub-menu performs its **action** and, for anything destructive, asks for confirmation.
- 4 Control always **loops back** to its parent menu — never a dead end.

An invalid option never crashes the program — it re-prints the same menu with a guidance message instead.



FEATURE — READ

Three Ways to View Stock Data

tampilkanData()

menampilkan()

filterKode()

filterHarga()

- **View all** — `menampilkan()` lists every active record (`hapus = False`) in a PrettyTable, or shows "Data Saham Kosong" if empty.
- **Filter by ticker** — `filterKode()` looks up one record by its 4-letter code, normalized with `.upper()`.
- **Filter by price** — `filterHarga()` matches any record whose opening *or* closing price equals the input.

Insight — the empty-dataset case is handled explicitly rather than left to crash, and every lookup normalizes its input so case never breaks a match.

menampilkan()

```
+-----+
|                                     |
|               RINGKASAN PASAR SAHAM KONOHA 2025               |
|-----+-----+-----+-----+-----+-----+
| Kode Saham | Nama Perusahaan | Harga Pembukaan | Harga Penutupan | Perubahan Harga | Nilai Transaksi |
|-----+-----+-----+-----+-----+-----+
| AADI      | Adaro Andalan Indonesia Tbk. | 6.550 | 6.800 | 3.82% | 44.460.882.500 |
| AALI      | Astra Agro Lestari Tbk. | 5.675 | 5.550 | 2.20% | 3.764.012.500 |
| AAPL      | Apple Inc. | 1.450 | 1.495 | 3.13% | 2.469.100.000.000 |
| AMZN      | Amazon.com Inc. | 3.400 | 3.450 | 1.47% | 1.740.500.000.000 |
| BBKA      | Bank Central Asia Tbk. | 8.275 | 7.900 | -4.53% | 736.670.570.000 |
| BBRI      | Bank Rakyat Indonesia (Persero) Tbk. | 3.930 | 5.550 | 41.23% | 941.219.074.000 |
+-----+-----+-----+-----+-----+-----+
...10 rows total
```

Captured from an actual run of `Manajemen Data Saham.py`.



FEATURE — CREATE

Adding a New Stock, Validated Step by Step

tambahData()

menambahkan()

cekPerusahaan()

konfersiKode()

cekPerubahan()

- 1 Check for a **duplicate ticker** first — if found, the existing entry is shown and the process stops.
- 2 `cekPerusahaan()` — company name must be exactly **4 words, letters only**.
- 3 `konfersiKode()` auto-generates the ticker from the **initials of those 4 words**.
- 4 Prices and value each require a **positive number** (`cekHargaPembukaan` , `cekHargaPenutupan` , `cekNilai`).
- 5 `cekPerubahan()` computes the **% change automatically** — never typed in by hand.

Insight

Auto-generating the ticker from the company name removes a whole class of typing mistakes.

Nothing is written to `dataSaham` until the preview is confirmed with **Ya**.

menambahkan()

```
Masukkan nama perusahaan      : Goto Gojek Tokopedia Indonesia
Masukkan harga pembukaan saham : 1200
Masukkan harga penutupan saham : 1250
Pergerakan Harga               : 4.17%
Data Baru:
```

Kode Saham	Nama Perusahaan	Harga Pembukaan	Harga Penutupan	Perubahan Harga	Nilai Transaksi
GGTI	Goto Gojek Tokopedia Indonesia	1.200	1.250	4.17%	500.000.000.000

```
⇒ Apakah Anda ingin menyimpan data baru tersebut? (Ya/Tidak): Ya
Data saham berhasil ditambahkan.
```

Captured from an actual run — ticker **GGTI** auto-generated from the 4-word company name.



FEATURE — UPDATE

Editing One Column, Safely

perbaruiData()

memperbarui()

- 1 Look up the record by **ticker**; if it doesn't exist, the process exits cleanly.
- 2 Choose exactly **one column** to edit: Company Name, Opening Price, Closing Price, % Change, or Transaction Value.
- 3 A copy (`dataAsli = saham.copy()`) is taken **before** editing — answering **Tidak** at the save prompt restores it untouched.
- 4 Editing % Change re-runs `cekPerubahan()` and flags moves over 10% as *"very significant"*.

Insight

The rollback copy means a cancelled edit never leaves the dataset half-changed. Editing the company name also regenerates the ticker, so the two fields never drift apart.

● ● ● memperbarui() — ticker BBKA

```
| Kode Saham | Nama Perusahaan | Harga Pembukaan | Harga Penutupan | Perubahan Harga | Nilai Transaksi |
| BBKA      | Bank Central Asia Tbk. | 8.275          | 7.900          | -4.53%         | 736.670.570.000 |
⇒ Masukkan kolom yang ingin diperbarui: 3 (Harga Penutupan)
Masukkan harga penutupan saham : 8200
Apakah data tersebut mau disimpan (Ya/Tidak)? Ya
Data saham PT Bank Central Asia Tbk. berhasil diperbarui
| BBKA      | Bank Central Asia Tbk. | 8.275          | 8.200          | -4.53%         | 736.670.570.000 |
```

Captured from an actual run — BBKA closing price updated live, 7.900 → 8.200.



FEATURE — DELETE

Deletion That Can Be Undone

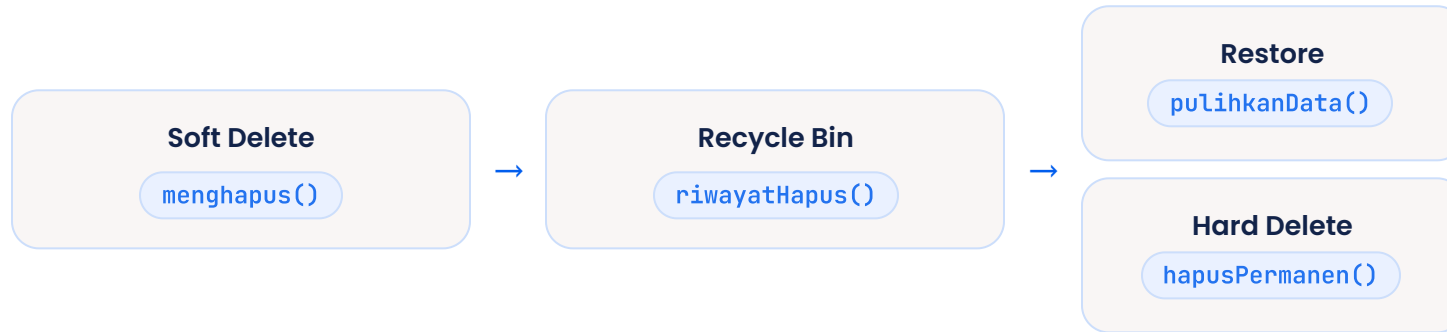
hapusData()

menghapus()

riwayatHapus()

pulihkanData()

hapusPermanen()



Insight

`hapus` is a flag, not a row removal — soft-deleted records simply stop appearing in `menampilkan()`.

Permanent deletion is a separate, explicit menu path — never triggered by accident.

● ● ● menghapus() → pulihkanData()

```
| AALI | Astra Agro Lestari Tbk. | 5.675 | 5.550 | 2.20% | 3.764.012.500 |
Apakah Anda yakin ingin menghapus data saham tersebut (Ya/Tidak)? Ya
Data saham berhasil dihapus. // AALI hidden, stored in recycle_bin
Masukkan kode saham yang ingin Anda pulihkan: AALI
Apakah ingin melanjutkan memulihkan data saham tersebut (Ya/Tidak)? Ya
Data saham berhasil dipulihkan. // AALI reappears in the active dataset
```

Captured from an actual run — ticker **AALI**, soft-deleted then restored.



VALIDATION & ERROR HANDLING

Every Input Loops Until It's Valid

Input	Rule	On invalid input
Company name	Exactly 4 words, letters only	Re-prompts with a guidance message
Opening / closing price	Integer > 0	<code>try/except ValueError</code> catches non-numeric input
Transaction value	Integer > 0	Same pattern as price fields
Menu choice	Must match a listed option	"Pilih menu yang valid dan coba lagi."
Ya / Tidak confirm	Normalized via <code>.capitalize()</code>	Anything else re-prompts the same question

```
cekHargaPembukaan()

def cekHargaPembukaan():
    while True:
        try:
            harga = int(input('Masukkan harga : '))
            if harga <= 0:
                print('Harga harus > 0.')
            else:
                return harga
        except ValueError:
            print('Input harus angka.')
```



PROGRAM DEMO

Three Real Runs of the Program

```
● ● ● Admin login

⇒ Masukkan kata sandi : ADMINSAHAM

[1] Menampilkan Data Saham
[2] Menambah Data Saham
[3] Memperbarui Data Saham
[4] Menghapus Data Saham
[5] Keluar
```

```
● ● ● New stock added

|  GGTI  |
| Goto Gojek Tokopedia |
| Indonesia |
|  1.200 → 1.250 |
|    4.17% |

Data saham berhasil
ditambahkan.
```

```
● ● ● Soft delete → restore

Ticker: AALI

Data saham berhasil
dihapus.
  ↓ recycle_bin
Data saham berhasil
dipulihkan.
```

All three panels are **captured from an actual run** of `Manajemen Data Saham.py` — not mockups.



CHALLENGES & SOLUTIONS

What Got in the Way — and How It Was Solved

Challenge	Solution
Formatting (<code>nominal()</code> , <code>presentase()</code>) turns numbers into strings	Arithmetic runs on raw ints/floats first — formatting happens only at display time
Deeply nested menus made it easy to lose track of "where" the user was	Every sub-menu is its own function that always returns control to its caller
An accidental delete could wipe out real data	Two-tier deletion: soft delete by default, hard delete as a separate confirmed action
Users can type almost anything into <code>input()</code>	Dedicated <code>cekXxx()</code> validators loop until the value is genuinely usable

Honest limitation: `dataSaham` and `recycle_bin` live only in memory — every change is lost when the program exits. Addressed in Recommendations.



CONCLUSION

What the Project Delivers

01

All **four CRUD operations** work end-to-end around a single, consistent stock dataset.

02

Two access levels (Investor / Admin) separate read-only browsing from data-changing actions.

03

A **recycle bin plus restore** path makes deletion reversible, not just soft.

04

Layered validation keeps every entry point — menus, prices, names — from accepting bad input.

05

A **structured terminal UI** (PrettyTable, centered banners, confirmation prompts) makes the CLI easy to follow.

4 / 4 CRUD operations · **2** access levels · **~30** functions across 949 lines of Python

RECOMMENDATIONS

Where This Project Goes Next

01

Persist data to CSV / JSON / SQLite instead of in-memory lists.

02

Replace the shared admin password with **per-user, hashed credentials**.

03

Add **price-range search** and column sorting.

04

Cover the validators and CRUD functions with **unit tests**.

05

Refactor the function-based design into **classes (OOP)**.

06

Migrate the interface to a **GUI or a Streamlit web app**.

Thank You